DOCUMENTO INTERNO · AGENTE IMPLEMENTADOR

Coder: del ticket de Jira al cambio revisado, con gates que no se saltan. Aplicado al proyecto Generic API del cliente Viva Aerobus

Coder conoce el sistema, conoce las reglas y sigue un proceso fijo de 12 fases. Analiza antes de escribir código, se detiene cuando el ticket no está listo y no publica nada sin la firma de una persona.

Tech and Solve
Cliente: Viva Aerobus · Proyecto: Generic API
12 fases · 2 etapas · 7 gates · 2 firmas humanas

DOCUMENTO CONFIDENCIAL. ELABORADO POR TECH AND SOLVE PARA USO INTERNO.

CONTENIDO

1. Qué es Coder
2. Qué habilita
3. Etapas y fases
4. Los gates
5. Rigor según el riesgo
6. Límites conocidos
7. Cómo empezar

Anexo: términos usados

1. Qué es Coder

Coder es un repositorio que vive al lado del repositorio de código, como carpeta hermana. **No contiene código de la aplicación**: contiene lo que un desarrollador necesita saber para cambiarla bien, el proceso para hacerlo y los mecanismos que obligan a seguirlo. Se opera abriendo una sesión de Claude Code dentro de esa carpeta.

Qué es	Un repositorio que reúne conocimiento del sistema, reglas de código y un proceso de 12 fases, operado por un agente de IA y vigilado por hooks.
Qué entrega	Un ticket convertido en un PR de un solo commit, validado y con evidencia; o, si el ticket no está listo, preguntas bloqueantes con opciones y una recomendación.
Quién decide	Las personas. El agente recomienda, pero no resuelve ambigüedades ni firma por nadie.

Pieza	Qué es	Quién la escribe
Conocimiento `documents/`	Qué es el sistema: conceptos, integraciones, mapa de dependencias, códigos de error, configuración. Cada línea cita el código y está anclada a un commit.	Solo el subagente `doc-sync`. Las ediciones a mano se descartan al iniciar sesión.
Reglas `guidelines/`	Cómo se debe escribir el código: 240 reglas en cinco categorías (estilo, arquitectura, robustez, proceso y dominio), extraídas de revisiones de PR reales.	El equipo, por su propio procedimiento y con evidencia de PR.
Proceso `process/`	Cómo se trabaja: un archivo por fase, la metodología de análisis y revisión, y plantillas de artefactos.	El equipo, por decisión de retro. Requiere una firma humana para editarse.
Vigilancia `.claude/`	Hooks, comandos y subagentes. Los hooks niegan acciones cuando falta un artefacto o una firma.	El equipo, igual que el proceso.
Trabajo `work/<CLAVE>/`	Los artefactos de cada ticket, con nombre fijo por fase. Se versionan: el trabajo se puede retomar desde cualquier máquina.	El agente, al terminar cada fase.
Automatización `tools/`	Creación de la tarea, puente con Jira, verificación de estilo y captura de evidencia de pruebas.	El equipo.

Las once reglas que no se rompen

El archivo orquestador de Coder fija once reglas. Las tres primeras sostienen todo el proceso; las demás nacieron de correcciones concretas en revisiones de PR.

Regla	Qué exige
1. Análisis antes de código	Las fases 0 a 4 se completan antes de escribir una línea. No hay atajo, ni en cambios triviales.
2. El conocimiento lo escribe solo el doc-sync	Si se detecta documentación faltante o incorrecta, se corre el doc-sync; nunca se corrige a mano.
3. Evidencia primero	Toda afirmación cita el código. Lo que no se encuentra se escribe como desconocido. Toda ambigüedad es una pregunta bloqueante, nunca una decisión del agente.
4. Nunca sobre conocimiento viejo	El repositorio se actualiza antes de cada pedido y la desactualización frente al código se mide en cada mensaje.
5. Las reglas gobiernan el cómo	El agente no inventa su propio estándar. Reutilizar lo que ya existe es parte de la regla.
6. Una sola vía para elegir ticket	«¿Qué ticket tomo?» se responde siempre con el mismo procedimiento sobre el tablero vivo.
7 a 10. Reglas de implementación	Credenciales de integración solo en la configuración de desarrollo; ninguna funcionalidad encendida desde el código; la lógica vive en el método compartido que la posee; leer la configuración donde ya está a mano.
11. Reglas sobre la base más reciente	Ningún cambio de reglas se confirma sobre una base vieja y ningún push sale sin rebase sobre la rama principal recién descargada.

2. Qué habilita

Coder atiende cinco tipos de pedido. Dos son de solo lectura; los otros tres escriben solo detrás de los gates de la sección 4 o de una aprobación explícita.

El desarrollador quiere	Cómo lo pide	Escribe
Empezar un ticket nuevo	`/implement API-9999` : crea la carpeta de trabajo, trae el ticket de Jira y recorre las 12 fases.	CON GATES
Retomar un ticket ya empezado	`/continue API-9999` : mira el ticket, el PR, la rama y la evidencia; propone un plan y espera aprobación antes de tocar nada.	CON APROBACIÓN
Defender un PR antes de pedir revisión	`/defend-changes API-9999` : un dossier que dice, por cada cambio, quién lo pidió, por qué así, sus efectos y un veredicto.	CON APROBACIÓN
Saber qué ticket tomar	Preguntarlo con sus palabras. Lee el tablero de Jira y aplica las reglas de elegibilidad.	SOLO LECTURA
Entender el sistema	Preguntar. Responde desde la base de conocimiento, con citas al código.	SOLO LECTURA

Capacidades que quedan disponibles

- **Trazabilidad numerada.** El plan numera sus pasos y sus escenarios de prueba; las pruebas ejecutan esa lista fila por fila y el pre-review la audita uno a uno.

- **Trabajo reanudable.** Cada fase cierra guardando y publicando la carpeta del ticket. La clave de Jira es el índice: del ticket se llega a la carpeta y de la carpeta al ticket.
- **Evidencia de pruebas para QA.** Un paquete por caso de prueba, con credenciales enmascaradas, que un subagente audita contra la matriz del ticket.
- **Ejecuciones de validación inmutables.** Si hay retrabajo, la validación anterior se archiva sin tocarla y los gates de publicación se vuelven a cerrar solos.
- **Varios tickets en paralelo.** El gate identifica la tarea por la rama activa del código, así que cada ticket se controla con sus propios artefactos.

Qué cambia para cada rol

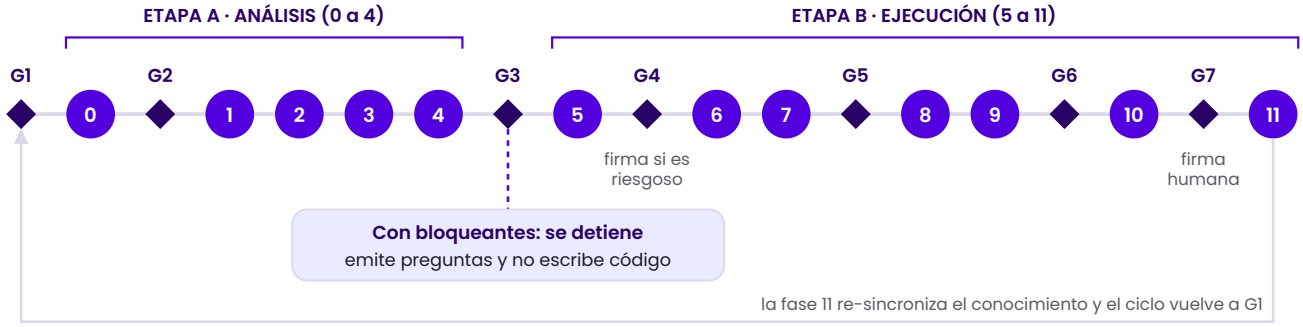
Desarrollador → **Coder** → **PO y tech lead** → **QA** → **Revisor del PR**

El desarrollador lanza el trabajo y firma; las demás personas responden, aprueban o revisan.

Actor	Qué hace	Qué deja de hacer
Desarrollador	Elige el ticket, lanza el comando, responde lo que el agente no puede decidir y firma los dos gates humanos.	Reconstruir a mano el contexto del sistema y armar desde cero la matriz de impacto.
Coder	Analiza con citas, escribe los artefactos de cada fase, implementa tras el veredicto verde, prueba, se auto-revisa y prepara el PR.	Nunca decide una ambigüedad, firma por una persona ni edita la base de conocimiento a mano.
PO y tech lead	Responden preguntas bloqueantes y aprueban el alcance y el plan.	Recibir preguntas abiertas: cada una llega con opciones, costo y recomendación.
QA	Recibe la evidencia por caso de prueba, ligada a la matriz del ticket.	Reconstruir qué se probó y pedir capturas después.
Revisor del PR	Revisa un único commit ya validado contra las 240 reglas, con una descripción completa.	Repetir correcciones que ya están escritas como regla.

3. Etapas y fases

Un ticket recorre dos etapas. La **etapa A, de análisis** (fases 0 a 4), usa la base de conocimiento y no toca el código. La **etapa B, de ejecución** (fases 5 a 11), solo empieza si el gate de la fase 4 da veredicto verde. Ninguna fase se salta: cada una tiene un criterio de salida y deja un artefacto con nombre fijo en `work/<CLAVE>/`.



Los círculos son fases y los rombos, gates. Un gate cerrado detiene el recorrido; G3 es el único que desvía a una salida sin código.

Etapa A: análisis

IN — **Ticket de Jira** G1 · CONOCIMIENTO VIGENTE

Antes de atender el pedido, Coder sincroniza su base de conocimiento con la rama principal del código.

0 — **Intake** G2 · TICKET LISTO

PREGUNTA QUE RESPONDE

¿Qué me piden realmente y por qué?

QUÉ HACE

Trae el ticket completo, verifica la Definition of Ready, clasifica el tipo de trabajo y el nivel de rigor, y separa lo que el ticket pide de la necesidad real.

Artefacto: `phase-00-intake.md`

1 — **Contraste con el código**

PREGUNTA QUE RESPONDE

¿Qué existe hoy, de verdad?

QUÉ HACE

Traza el flujo de punta a punta, reproduce los bugs, revisa la historia de git y clasifica las discrepancias entre lo que el ticket supone y lo que el código hace.

Artefacto: `phase-01-contrast.md`, con el ancla del conocimiento usado.

2 — **Radio de impacto**

PREGUNTA QUE RESPONDE

¿Qué se mueve si toco esto?

QUÉ HACE

Recorre código, contratos, datos, aspectos no funcionales y operación con el mapa de dependencias inverso. Arma la matriz de impacto, que otra persona valida en diez minutos.

Artefacto: `phase-02-impact-matrix.md`

3

Cobertura y viabilidad

PREGUNTA QUE RESPONDE

¿El ticket y sus hermanos cubren todo? ¿Se puede hacer?

QUÉ HACE

Lista los huecos, los mapea a tickets, ordena las dependencias y concluye la viabilidad en cinco ejes: técnica, datos, dependencias, alcance y riesgo.

Artefacto: `phase-03-feasibility.md`

4

Bloqueantes y supuestos G3 · VEREDICTO

PREGUNTA QUE RESPONDE

¿Qué no puedo decidir solo?

QUÉ HACE

Clasifica las dudas, redacta cada pregunta bloqueante con opciones y una recomendación, registra los supuestos con su forma de validarlos y emite el veredicto.

Artefacto: `phase-04-verdict.md`, con una única línea de veredicto.

Etapa B: ejecución

5

Planeación G4 · ESCRITURA

PREGUNTA QUE RESPONDE

¿Cómo lo hago, en qué orden y con qué riesgos?

QUÉ HACE

Compara al menos dos opciones y escribe los planes de datos, compatibilidad, flag, observabilidad, pruebas, despliegue y reversa, y seguridad. Numera los pasos y los escenarios de prueba, y cita las reglas que debe honrar.

Artefacto: `phase-05-plan.md`

6

Implementación

PREGUNTA QUE RESPONDE

¿Lo construyo según el plan, sin desviarme en silencio?

QUÉ HACE

Trabaja en una rama ligada a la clave del ticket. En los bugs escribe primero el test que falla, reutiliza lo que ya existe y aplica el estilo de código del equipo al final. Lo que esté fuera de alcance se convierte en un ticket nuevo; una sorpresa grande lo devuelve a la fase 4.

Artefacto: `phase-06-code-style.md`

7

Testing G5 · PRUEBAS

PREGUNTA QUE RESPONDE

¿Funciona y no rompí nada del radio de impacto?

QUÉ HACE

Ejecuta los escenarios fila por fila y corre la suite completa. Arma el paquete de evidencia por caso de prueba, que audita un subagente.

Artefacto: `phase-07-testing.md` y `evidence/`

8

Preparación de la liberación

PREGUNTA QUE RESPONDE

¿Se puede desplegar y revertir de forma segura?

QUÉ HACE

Escribe el runbook y prepara la configuración por entorno, el kill switch, la reversa, el smoke test y las notas de liberación.

9

Pre-review G6 · PRE-REVIEW

PREGUNTA QUE RESPONDE

¿Está listo para gastar el tiempo de otra persona?

QUÉ HACE

Revisa el diff línea por línea y lo valida contra las reglas. Comprueba que el ticket, el plan y el código cuadran, audita los desvíos y deja un solo commit.

Artefacto: `phase-09-pre-review.md`

10

PR y revisión G7 · PUBLICACIÓN

PREGUNTA QUE RESPONDE

¿Cómo transfiero el contexto y obtengo la aprobación?

QUÉ HACE

Muestra el resumen de publicación y espera la firma. Abre el PR con la plantilla del repositorio y entrega a Jira, siempre con una simulación previa.

11

Post-merge y cierre

PREGUNTA QUE RESPONDE

¿Funcionó en producción y quedó cerrado el ciclo?

QUÉ HACE

Verifica el despliegue, enciende el flag de forma progresiva y monitorea. Re-sincroniza el conocimiento con el nuevo código, cierra el ticket, limpia y registra el aprendizaje en una retro de proceso.

✓

Cambio en producción y conocimiento al día

El siguiente ticket parte de una base de conocimiento que ya incluye este cambio.

4. Los gates

Un gate es un punto del recorrido que **solo se cruza cuando existe una evidencia concreta**: un archivo con nombre fijo y una línea marcadora al inicio de una línea. Los hooks de Coder leen esos archivos antes de cada acción y la niegan si falta algo. Agrupamos los controles en siete gates; dos de ellos exigen una firma que solo una persona puede crear.

Gate	Qué verifica	Qué desbloquea	Quién lo abre
G1 · Conocimiento vigente antes de cada pedido	Que la base de conocimiento esté al día con la rama principal del código y que existan las carpetas de conocimiento y de reglas.	Atender el pedido. Sin conocimiento ni reglas presentes se niega toda escritura y publicación; con conocimiento atrasado, el turno no puede cerrar.	HOOK
G2 · Ticket listo fase 0	La Definition of Ready: el ticket tiene la etiqueta de casos de prueba listos y una matriz de casos en su subtarea de evidencia.	Iniciar el análisis. Si falla, el ticket no se inicia y se pregunta al solicitante.	HERRAMIENTA
G3 · Veredicto fase 4	Una única línea de veredicto: listo para implementar, requiere definición, o no implementable como está.	Pasar a la etapa B. Con cualquier veredicto distinto del verde, el agente se detiene y emite las preguntas.	AGENTE
G4 · Escritura fin de la fase 5	Los artefactos de las fases 0 a 5, el ancla del conocimiento, el veredicto verde y, si el ticket es riesgoso, la firma `HUMAN-GATE-OK`.	Escribir en el repositorio del código.	PERSONA SI ES RIESGOSO
G5 · Pruebas fase 7	La línea de suite en verde, con la salida completa de la suite.	Junto con G6 y G7, el push y el PR.	AGENTE
G6 · Pre-review fase 9	Estilo verificado; revisión contra las reglas aprobada; completitud entre ticket, plan y código; desvíos nulos o aprobados. Cada aprobación queda atada al commit exacto que validó.	Junto con G5 y G7, el push y el PR. Un commit posterior anula la aprobación.	AGENTE
G7 · Publicación fase 10	La firma `PUSH-APPROVED`, creada a mano por el desarrollador después de ver el resumen de publicación.	`git push`, `gh pr create` y la entrega a Jira.	PERSONA

El gate central: el veredicto de la fase 4

Veredicto	Significa	Qué hace el agente
LISTO **Ready to implement**	Objetivo, contratos, criterios de aceptación e impacto claros; cero bloqueantes duros.	Continúa a la fase 5.
DEFINIR **Needs definition**	Se puede implementar, pero hay bloqueantes sin definir.	Se detiene. Emite las preguntas con opciones y recomendación, y pide marcar el ticket como bloqueado. No escribe código ni plan.
DETENER **Not implementable**	La fuente de verdad está fuera del repositorio, depende de algo no construido o contradice una regla existente.	Se detiene. Explica por qué y propone dos o tres alternativas con una recomendación.

El agente separa dos tipos de información faltante: el **hueco de negocio**, que es una pregunta para el solicitante, y el **hueco de verificación**, que es trabajo propio en el código y no se le traslada al negocio.

Lo que el desarrollador ve en cada mensaje

El estado de los gates se inyecta en cada mensaje de la sesión. Así, ni el agente ni el desarrollador pierden de vista en qué fase va el ticket.

Estado del pipeline · API-9999 · ejemplo

G1 · Conocimiento	OK	Anclado al último commit de la rama principal.
G3 · Veredicto	OK	Listo para implementar.
G4 · Escritura	OK	Plan aprobado; ticket de nivel normal.
G5 · Pruebas	OK	Suite completa en verde.
G6 · Pre-review	BLOQUEA	La revisión contra las reglas pide cambios: dos hallazgos.
G7 · Publicación	BLOQUEA	Falta la firma del desarrollador.

El push y el PR quedan negados hasta cerrar G6 y G7.

Ejemplo ilustrativo del tipo de información; no reproduce el formato exacto del hook.

Reglas de integridad de los gates

- **El artefacto se escribe al terminar el trabajo, nunca antes.** Escribir el marcador sin hacer el trabajo es falsificar el gate.
- **Las firmas humanas están negadas al agente.** Los hooks rechazan cualquier intento de crear `HUMAN-GATE-OK`, `PUSH-APPROVED` o el permiso para cambiar el proceso.
- **Los pasos humanos no se dan por hechos.** Cada uno queda registrado como hecho por una persona (quién y cuándo), entregado y pendiente, o no aplica, con su razón.
- **Las firmas no se versionan.** Una firma confirmada en git abriría el gate en la copia de otra persona que no aprobó nada.

5. Rigor según el riesgo

El nivel se decide en la fase 0 y se escribe en el ticket; si hay duda, se sube un nivel. El nivel cambia la profundidad de los artefactos, nunca el contrato: **el gate de la fase 4 aplica siempre** y los hooks exigen todos los archivos en cualquier nivel.

Nivel	Ejemplos	Qué aplica
Trivial	Texto, errata, nivel de log.	Las fases 0, 1, 4, 6, 7, 9 y 10 completas. Los artefactos de las fases 2, 3 y 5 se reducen a un párrafo que explica por qué.
Normal	Funcionalidad acotada, bug sin impacto en datos.	Todas las fases. El registro de decisión de arquitectura y el runbook son opcionales.
Riesgoso	Migración, contrato, pagos, flags, varios repositorios.	Todas las fases, completas: registro de decisión, runbook y reversa probada, más la firma humana sobre el plan (G4).

Un **incidente en producción** sigue una ruta comprimida: primero mitigar, después un arreglo mínimo y reversible revisado por al menos una persona. Nada se elimina, se posterga: en las 48 horas siguientes se exigen el análisis completo, los tests que faltaron y la re-sincronización del conocimiento.

6. Límites conocidos

Los gates son un control determinista, no una garantía de calidad. Estos son sus límites y lo que los compensa.

Límite	Qué lo compensa
Los hooks verifican que el artefacto y su marcador existan, no su calidad.	La regla de integridad, la revisión contra las reglas de la fase 9, la validación cruzada de la matriz y la revisión humana del PR.
Es un control, no un entorno aislado.	Las capas externas siguen siendo la revisión del PR y los permisos de GitHub.
El gate de conocimiento deja pasar cuando sincronizar es imposible: sin ancla, sin el repositorio de código o con la descarga fallida.	El análisis declara con qué ancla trabajó y si estaba desactualizado. El gate solo se puede apagar por sesión, y eso se dice en la respuesta.
El pipeline solo corre desde Claude Code.	Las sesiones de Copilot quedan en solo lectura con su propio adaptador.
Supone dos repositorios hermanos en la misma carpeta.	Si el supuesto no se cumple, el agente se detiene y pide las rutas reales.

7. Cómo empezar

Clonar Coder junto al repositorio de código, abrir una sesión de Claude Code dentro de la carpeta de Coder y autenticar una vez la conexión con Jira (`/mcp` , luego `atlassian`). Desde ahí, `/implement` con la clave del ticket arranca la fase 0. El progreso queda en `work/<CLAVE>/` y se retoma en cualquier momento con el mismo comando.

Fuente: repositorio `VivaAerobus.Generic.ApiLLMCoder` , rama `main` , commit `4b2cdd4` del 21 de octubre de 2026: `README.md` , `CLAUDE.md` , `process/` y `.claude/hooks/lib-gate.sh` . Consultado el 27 de octubre de 2026.

Anexo: términos usados

Término	Qué significa
Gate	Punto del recorrido que solo se cruza cuando existe una evidencia concreta.
Hook	Script que Claude Code ejecuta antes o después de una acción del agente y que puede negarla.
Artefacto	Archivo con nombre fijo que una fase deja en la carpeta del ticket.
Marcador	Línea exacta al inicio de una línea de un artefacto que el hook lee para abrir un gate.
Firma humana	Archivo que solo una persona puede crear a mano y que abre un gate; al agente le está negado.
Doc-sync	Subagente que actualiza la base de conocimiento a partir de los cambios del código.
Ancla	Commit del código a partir del cual se escribió la base de conocimiento.
Subagente	Agente auxiliar con una tarea acotada y sus propias reglas; no hereda el contexto del agente principal.
Definition of Ready	Condiciones mínimas que un ticket cumple antes de poder empezarse.
Radio de impacto	Todo lo que se mueve, directa o indirectamente, al cambiar una parte del sistema.
Matriz de impacto	Tabla con una fila por elemento afectado, su riesgo, su dueño y si el ticket lo cubre.
Pre-review	Auto-revisión del diff, antes de pedir la revisión de otra persona.
Squash	Unir todos los commits de un cambio en uno solo.
Feature flag y kill switch	Interruptor de configuración que enciende o apaga una funcionalidad sin desplegar.
Runbook	Lista de pasos exacta para desplegar y revertir un cambio.
Smoke test	Verificación corta en producción de que lo esencial funciona tras el despliegue.
MCP	Protocolo con el que Claude Code se conecta a servicios externos, como Jira.